# Sprint 4 · Python para IA

*¿cómo automatizamos lo que hemos hecho a mano?*

**FECHAS**
27–29/10

**HORAS**
3 h de teoría (T) · 5 h de práctica (P) · 2 h de proyecto (PBL) = 10 h

## Qué aprenderás

1. A trabajar en Colab o Jupyter y a escribir Python básico: variables, tipos, decisiones y bucles.
2. A crear **funciones** y **módulos** reutilizables, con documentación y pruebas.
3. A manejar **listas, diccionarios, conjuntos** y comprensiones para representar datos reales.
4. A leer y escribir **CSV y JSON** con la biblioteca estándar y a controlar errores con excepciones.
5. A organizar tu código con clases, buenas prácticas y **Git básico**, y a entregar una herramienta con pruebas.

## Requisitos previos

- Cuenta de Google para usar Colab (o Python 3.11 y Jupyter/VS Code en tu equipo).
- **No hace falta saber programar.** Si vienes de Java, en los cuadernos hay comparativas con Python.
- Los datos (`reservas_turisdata.csv`) los descargan los cuadernos solos; para el reto los tendrás también en el repositorio.
- Recuerda: los datos son **sintéticos** (no son estadísticas reales del turismo canario).

## Prácticas (5 h)

Ejecuta los cuadernos en orden. Cada ejercicio se **autocorrige**: si la celda de comprobación falla, lee el mensaje, corrige y vuelve a ejecutar.

| Cuaderno | Tema | Tiempo |
|---|---|---|
| `S04_01_entorno_sintaxis` | Entorno, variables, tipos, operadores, `if`, `for`, `while` | 60 min |
| `S04_02_funciones_modulos` | Funciones, parámetros, `*args`, `lambda`, módulos | 60 min |
| `S04_03_estructuras_datos` | Listas, tuplas, diccionarios, conjuntos, comprensiones | 60 min |
| `S04_04_ficheros_excepciones` | CSV y JSON, `try/except`, lector robusto | 60 min |
| `S04_05_poo_buenas_practicas` | Clases, herencia, buenas prácticas, Git básico | 60 min |

## RETO Reto PBL · «Herramientas internas» (2 h)

**En pocas palabras (léelo primero)**

**Qué os pide el cliente.** El equipo de reservas de TurisData calcula sus cifras semanales **a mano en una hoja de cálculo**. Es lento y cada persona lo hace un poco distinto. Os pide una **herramienta interna**: un pequeño programa que **lea el fichero de reservas, calcule tres cifras importantes (KPIs), avise con claridad si el fichero está mal y guarde los resultados** en un formato que otros sistemas puedan leer.

**Qué es un KPI.** Una cifra que resume cómo va el negocio. Aquí son tres: la **ocupación aproximada**, el **ingreso medio por reserva** y la **tasa de cancelación por canal**.

**No hace falta tener experiencia previa de programación.** Os damos una **base casi hecha** (una plantilla con funciones vacías y pruebas) y vuestro trabajo es **rellenarla paso a paso**, funcionando cada vez que una prueba pasa de rojo a verde.

**Qué vais a hacer, en cinco pasos:**

1. **Copiar la carpeta de la plantilla** (`recursos/reto_herramientas/`) y ejecutar las pruebas (al principio fallan; es lo normal).
2. **Rellenar las funciones** una a una hasta que sus pruebas pasen.
3. **Ejecutar el programa con el fichero real** y comprobar el resultado.
4. **Provocar errores a propósito** (fichero que no existe, columna que falta) y ver que el programa **informa con claridad** en vez de romperse.
5. **Escribir el README** y subirlo todo a Git con varios *commits*.

**Qué vais a entregar:** un repositorio Git con `kpis.py`, `test_kpis.py`, `README.md` y un `salida.json` generado con el CSV real.

**Qué vais a aprender haciéndolo:** a convertir un trabajo manual en una herramienta **repetible**, a **probar** vuestro código y a usar **Git**.

**Cuánto tiempo:** 2 horas dentro del sprint (27–29/10).

### Palabras que van a aparecer en este reto

| Palabra | Qué significa, en sencillo |
|---|---|
| **Script** | Un programa corto escrito en un fichero (aquí, `kpis.py`) |
| **Función** | Un bloque de código con un nombre que hace una tarea concreta y se puede reutilizar |
| **CSV** | Un fichero de texto con una tabla, con los valores separados por comas |
| **JSON** | Otro formato de texto para guardar datos, muy usado para que se entiendan distintos programas |
| **Biblioteca estándar** | Herramientas que ya vienen con Python (`csv`, `json`). No hace falta instalar nada |
| **Prueba (test)** | Un trozo de código que comprueba que otro funciona. Usa `assert`: «esto tiene que ser verdad» |
| **Excepción** | Un error que ocurre mientras el programa se ejecuta. Se puede **controlar** para evitar que el programa se rompa |
| **Docstring** | El texto explicativo que escribís justo dentro de una función entre triples comillas |
| **PEP 8** | La guía de estilo de Python: cómo poner nombres, sangría y espacios |
| **Commit** | Guardar en Git una versión de vuestro trabajo con un mensaje que dice qué habéis cambiado |
| **Proxy** | Una medida aproximada, usada cuando no se puede medir lo exacto |

### Contexto del cliente

El equipo de reservas de *TurisData Canarias* calcula cada semana sus cifras a mano en una hoja de cálculo: es lento y cada persona lo hace un poco distinto. Os pide una **herramienta interna**: un script que lea el CSV de reservas, calcule tres KPIs, avise con claridad si el fichero está mal y deje los resultados en un JSON que otros sistemas puedan leer.

### Qué tenéis que entregar

Un **repositorio Git** con:

- `kpis.py`: el script (funciones + `main`), solo con la biblioteca estándar (`csv`, `json`…).
- `test_kpis.py`: las pruebas con `assert` (os damos una base; podéis añadir las vuestras).
- `README.md`: cómo se ejecuta, cómo se prueba, definición de cada KPI y decisiones (plantilla en `recursos/reto_herramientas/README_PLANTILLA.md`).
- Un `salida.json` generado con el CSV real.
- *(Opcional)* `kpis_pandas.py`: la misma lógica con pandas.

**KPIs:** ocupación aproximada (*proxy*), ingreso medio por reserva y tasa de cancelación por canal. Las definiciones exactas están en los docstrings de la plantilla.

### Restricciones

- **Tiempo:** 2 h dentro del sprint.
- Versión principal solo con la **biblioteca estándar**; pandas solo en la parte opcional.
- El script **no puede romperse** con un fichero inexistente, con columnas que faltan o con filas defectuosas: debe informar.
- Sin datos personales reales ni claves en el repositorio.

### Criterios de evaluación (rúbrica del curso, 0–4 en cada criterio)

| Criterio | Qué se mirará en este reto |
|---|---|
| **Corrección técnica** | Las pruebas pasan, los KPIs coinciden con los esperados, los errores están controlados. |
| **Reproducibilidad** | Otra persona clona el repositorio, sigue el README y obtiene el mismo JSON. |
| **Análisis y comunicación al cliente** | El README y el JSON se entienden sin ser programador; cada KPI está definido y sus límites explicados (por qué la ocupación es solo un proxy). |
| **IA responsable** | Se explica qué parte se ha hecho con ayuda de IA y qué se ha revisado; se mencionan los límites de datos sintéticos y del proxy. |

**Pasos sugeridos**

- **Día 1 (mitad del tiempo):** copiad la carpeta `recursos/reto_herramientas/`, ejecutad `python test_kpis.py`, implementad `leer_reservas` y las funciones de KPI una a una hasta que pasen las pruebas.
- **Día 2:** `exportar_json` y `main`, ejecutad con el CSV real, escribid el README y haced los *commits* (`git init`, `git add`, `git commit`). Opcional: versión pandas.

**Guía paso a paso para principiantes**

***Paso 1 · Preparar la carpeta y ver que las pruebas fallan (día 1, 20 minutos)***

1. Descargad la carpeta `recursos/reto_herramientas/` (viene con `kpis.py`, `test_kpis.py` y `README_PLANTILLA.md`) y el fichero `reservas_turisdata.csv` (carpeta `datos/` del repositorio del curso). Dejadlos en la misma carpeta.
2. Abrid una terminal o un cuaderno de Colab en esa carpeta y ejecutad `python test_kpis.py`.
3. **Veréis errores o «pruebas no superadas».** Es normal: las funciones están vacías.
4. Repartid las funciones entre las personas del equipo: cada una trabaja una y, cuando pasa, otra la revisa.

***Paso 2 · Rellenar las funciones, de una en una (día 1, 60–70 minutos)***

Orden recomendado:

1. `leer_reservas(ruta)`: lee el CSV y devuelve la lista de reservas y los avisos. Debe **descartar y contar las filas defectuosas** (por ejemplo, un precio que no es un número) y guardar un aviso por cada una. No debe romperse.
2. `tasa_cancelacion_por_canal(reservas)`: para cada canal, `cancelaciones ÷ reservas × 100`, con 1 decimal.
3. `ingreso_medio_por_reserva(reservas)`: media de `noches × precio_noche` de las reservas **no canceladas**, con 2 decimales.
4. `ocupacion_proxy(reservas, capacidad_habitaciones)`: noches de reservas no canceladas ÷ (días del periodo × habitaciones) × 100, como explica el docstring. **Es una aproximación** (todas las noches se cuentan en el periodo aunque la estancia lo desborde); explicad en el README por qué solo es un proxy.

**Método de trabajo (muy importante):** programad **una función**, ejecutad `python test_kpis.py`, mirad qué prueba falla, corregid, repetid. Es lo que hacen los profesionales: se llama trabajar «guiado por pruebas».

***Paso 3 · Exportar el resultado y hacer funcionar el programa entero (día 2, 30 minutos)***

1. `exportar_json(datos, ruta)`: guarda el resultado en un fichero JSON con codificación UTF-8 y sangría de 2 espacios.
2. `main(...)`: lo une todo. Ejecutad: `python kpis.py reservas_turisdata.csv salida.json --capacidad 40`
3. Abrid `salida.json` y comprobad que contiene los tres KPIs y la sección `calidad` (con las filas descartadas).

***Paso 4 · Probar que no se rompe (día 2, 15 minutos)***

Provocad errores **a propósito** y comprobad que el programa **da un mensaje claro** y no un montón de texto rojo:

- Un fichero que no existe.
- Un CSV al que le falta una columna (haced una copia y quitadla).
- Una fila con un dato incorrecto.

Un mensaje bueno: *«No se encuentra el fichero reservas.csv»*. Un mensaje malo: *«Traceback (most recent call last)…»*.

***Paso 5 · README y Git (día 2, 30 minutos)***

1. Copiad `README_PLANTILLA.md` como `README.md` y rellenad: qué hace, cómo se ejecuta, cómo se prueba, definición de cada KPI y decisiones (**por qué la ocupación es solo un proxy**).
2. En la terminal: `git init`, `git add .`, `git commit -m "Primera versión"`. Haced **al menos 3 commits con mensajes claros** (por ejemplo, «Añade lectura del CSV», «Añade KPIs», «Añade README»). Cada persona debe hacer alguno.
3. Si lo subís a GitHub, hacedlo en la carpeta `sprint-04/` de vuestro repositorio de equipo.
4. *(Opcional)* Creád `kpis_pandas.py` con la misma lógica usando pandas y comparad.

**Cómo usar la IA en este reto**

La IA puede ahorrar mucho tiempo en programación, pero **hay una trampa**: te da código que parece funcionar y no lo entiendes. Por eso aquí hay reglas claras.

| Para qué | Ejemplo de petición | Qué tenéis que comprobar |
|---|---|---|
| Entender un error | «Me sale este error al ejecutar mi script: *[pega el mensaje completo]*. Explícame qué significa, sin darme todavía la solución» | Que lo entendéis y lo arregláis vosotros |
| Entender una función | «Explícame línea por línea qué hace esta función: *[código]*» | Ejecutadla con un ejemplo pequeño |

| Para qué | Ejemplo de petición | Qué tenéis que comprobar |
|---|---|---|
| Pedir una pista | «Quiero calcular el porcentaje de cancelaciones por canal con la biblioteca estándar de Python. ¿Qué estructura de datos me conviene? No me des el código completo» | Que respeta la restricción de **solo biblioteca estándar** |
| Mejorar el estilo | «Revisa si este código sigue PEP 8 y dime qué cambiarías» | Que las pruebas siguen pasando después |

**Reglas del reto:**

- **Todo lo que la IA os dé, lo ejecutáis y lo entendéis.** Si el docente os pregunta por una línea, tenéis que saber explicarla.
- **Las pruebas (`test_kpis.py`) mandan:** si el código de la IA no las pasa, no vale.
- **Declarad el uso** en la sección «Uso de IA y límites» del README: *qué le pedisteis y qué revisasteis*.
- No peguéis **datos personales ni claves** en una IA.

### Errores frecuentes (y cómo evitarlos)

| Error | Por qué ocurre | Cómo evitarlo |
|---|---|---|
| El programa se rompe con un fichero mal | No se controlan las excepciones | Usad `try` / `except` y mensajes claros |
| La media sale rara | Se incluyen reservas canceladas en el ingreso | Releed la definición: **solo las no canceladas** |
| Cambiar las pruebas para que pasen | Prisa | **No toquéis** las pruebas dadas; añadid las vuestras aparte |
| Un solo *commit* gigante | «Lo subo al final» | Haced commits pequeños mientras avanzáis |
| README vacío o copiado | Se deja para el final | Escribid cada sección cuando terminéis esa parte |
| Nombres de funciones o variables raros (`x1`, `aaa`) | Prisa | Nombres que se entiendan: `tasa_cancelacion_por_canal` |

### Un buen resultado se parece a...

- **Salida del programa:** un JSON ordenado como:

```
{
  "reservas_analizadas": 1996,
  "ocupacion_proxy_pct": 61.3,
  "ingreso_medio_por_reserva": 412.87,
  "tasa_cancelacion_por_canal_pct": {"directo": 18.5, "web": 24.1},
  "calidad": {"filas_validas": 1996, "filas_descartadas": 4, "avisos": ["fila 57: precio no numérico"]}
}
```

(Las cifras son un ejemplo; las vuestras saldrán del CSV real.)

- **Mensaje de error:** «Falta la columna `precio_noche` en el fichero».
- **README:** alguien que no es del equipo puede ejecutarlo siguiendo solo las instrucciones.

### Antes de entregar: autoevaluación rápida

| Pregunta | Sí / No |
|---|---|
| ¿`python test_kpis.py` termina con todas las pruebas superadas? | |
| ¿`python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los tres KPIs y la sección `calidad`? | |
| ¿El programa informa con claridad si falta el fichero o una columna? | |
| ¿Todas las funciones tienen docstring y siguen PEP 8? | |
| ¿El README explica cómo ejecutar, cómo probar y qué significa cada KPI? | |
| ¿Hay al menos 3 commits con mensajes claros y sin claves ni datos personales? | |
| ¿Otra persona ha revisado el código con la rúbrica? | |

**Si os atascáis**

- **No sabemos por dónde empezar con Python:** repasad el cuaderno `S04_04_ficheros_excepciones` (lectura de CSV y excepciones).
- **Una prueba falla y no entendemos por qué:** leed el mensaje de la prueba (dice qué esperaba y qué recibió) y probad la función en una celda con un ejemplo pequeño.
- **Git no nos funciona:** repasad el último cuaderno (`S04_05`) o pedid ayuda al docente; podéis subir los ficheros desde la web de GitHub.
- Más ayuda: Cómo se trabaja un reto.

**Qué se enseña en la review**

Una demo de 3–5 minutos: ejecutar el script con el CSV real, mostrar el JSON, provocar un error a propósito (fichero inexistente) y enseñar el mensaje, y mostrar las pruebas en verde. Explicad una decisión de diseño (por ejemplo, cómo definisteis la ocupación).

## Definition of Done

- [ ] `python test_kpis.py` termina con todas las pruebas superadas.
- [ ] `python kpis.py reservas_turisdata.csv salida.json` genera el JSON con los tres KPIs y la sección `calidad`.
- [ ] El script informa con un mensaje claro si falta el fichero o una columna, y descarta y cuenta las filas defectuosas.
- [ ] Todas las funciones tienen docstring y siguen PEP 8 (nombres, sangría).
- [ ] El README explica cómo ejecutar, cómo probar y las definiciones de los KPIs.
- [ ] El repositorio tiene al menos 3 *commits* con mensajes claros y no contiene claves ni datos personales.
- [ ] Alguien del grupo ha revisado el código con la rúbrica.

## Recursos

- Tutorial oficial de Python (en español): https://docs.python.org/es/3/tutorial/
- Módulo `csv`: https://docs.python.org/es/3/library/csv.html · Módulo `json`: https://docs.python.org/es/3/library/json.html
- PEP 8, guía de estilo (resumen): https://peps.python.org/pep-0008/
- Git, guía básica: https://git-scm.com/book/es/v2
- Google Colab, primeros pasos: https://colab.research.google.com/